

Saturating Benchmarks

Mapping the Frontier of Reducible Decidability in Practice

Christoph Kirsch

University of Salzburg, Austria

Czech Technical University, Prague, Czechia

ck@cs.uni-salzburg.at

Abstract

Present a platform any benchmark whose questions reduce to decision procedures. What would it mean for the platform to *finish* the benchmark — and what architecture makes finishing detectable, trustworthy, and portable across domains? We define *saturation*: every question either answered along every feasible route, with measured cost and quantified trust, or open on the books with its first failing obstacle, the evidence that would move it, and the exact instrument — a *frontier pair*, a pair whose design is unknown but whose required contract and design evidence are derived mechanically — that would close it, or the stated reason none can. We prove that one architecture suffices: answerability is a monotone filtration of candidate routes through five ordered conditions, so exploration never loses ground, every open question has a unique first-failing obstacle that only advances, and N adequately targeted extensions answer any question answerable at all — relative completeness in the lineage of Cook. The compositional core is mechanized in Lean 4.

The instrument exists: hurdy-gurdy, a platform of deterministic, fidelity-graded translation pairs with untrusted authors and measured trust, is the means to this end, and this paper states the end. We give the frontier problem, the explorer architecture, the facilitation theorems with their honest assumptions, the domain kit any field must supply, and the pre-registered saturation protocol for a benchmark designed by others. This is a working draft: measurements are deliberately absent — the benchmarks section of this paper is the saturation report its protocol will produce.

1 Introduction

Present a platform any benchmark whose questions reduce to decision procedures; the platform eventually learns all ways feasible in practice to solve it, and saves, as structured evidence, everything not yet solvable and why. The instrument that makes such a claim checkable — deterministic, fidelity-graded translation pairs with untrusted authors — exists and is measured elsewhere [4]; this paper states what it is *for*, and proves that it can do it. Section 2 defines the problem; Section 5 proves the six properties an explorer needs; the sections between compress the instrument and its loop to exactly what those theorems consume.

2 The frontier problem

Domains. A *domain* is where questions live and decisions happen. Its signature supplies, and nothing else about it matters: languages, each carrying a *formal semantics* — a definable meaning function over its programs, exposing named *observables* — with formal semantics the only admission criterion; among them some *reasoning languages*, each owning mechanized decision procedures with declared budgets and, where available, certificate checkers; and a supply of independent *semantic anchors* — external formalizations against which a translation can be checked by something its author did not write. Hardware model checking is a domain; C program analysis is a domain; chemical reaction networks are a domain. The theorems of Section 5 use only the signature, which is what *in any given domain* will mean.

Questions. A *question* $q = (p, \varphi)$ is a program p in one of the domain’s languages and a condition φ over p ’s observables, asked existentially (is φ reachable?) or universally (does φ hold for all inputs, within declared bounds?). A *benchmark* B is a pinned, finite set of questions with recorded provenance — pinned, so that “the benchmark” names the same set at every loop iteration.

Items, registries, candidates. The unit of admission is the *item*: either a *pair* — a deterministic, contract-carrying translation edge between two languages [4] — or a language-attached *capability*: a solver, a checker, an anchor. A *registry* G is the set of admitted items; growth is additive (items are admitted, never silently removed), and every admission is gated. A *candidate* answer for q rests on finitely many items: the route’s hops plus the destination capabilities it invokes — one list. Pairs for edges, capabilities for nodes: a candidate is the route-level object, and it is the only object the definitions below need.

Definition 2.1 (Answerability). Fix, per domain, N *conditions* on candidates, checked in a fixed *diagnosis order*. hurdy-gurdy instantiates $N = 5$: *connectivity* (the candidate is a route from p ’s language to a reasoning language), *loss* (φ ’s observables survive the route’s composed projection), *shape* (φ ’s logical form is one the destination’s procedures decide), *cost* (a procedure returns a verdict other than *unknown/resource-out* within its declared budget), and *trust* (the answer meets the asker’s assurance floor). Write $S_k(G, q)$ for the candidates available in G that survive the

first k conditions — the *answerability filtration*, shrinking as k grows and growing as G grows. Then q is *answerable at G* iff $S_N(G, q) \neq \emptyset$.

Two words in the title are fixed by this definition. *In practice*: the cost condition quantifies over declared budgets and measured runs, never over asymptotics — *resource-out* is a permanent verdict class, and a route without a measurement reads *unmeasured*, never cheap. *Reducible*: answerability quantifies over candidates built from admitted items, so the answerable set is the closure of the domain’s questions under the admitted reductions — not an oracle’s reach.

Definition 2.2 (Diagnosis). For q unanswerable at G , the *diagnosis* is the first failing condition: the k with $S_k(G, q) \neq \emptyset = S_{k+1}(G, q)$. Theorem 5.3 shows it exists, is unique, and under growth only advances — “the obstacle” is honest vocabulary, and it is the frontier’s local gradient: which kind of instrument would move the map *here*.

The currency: three tiers of pair. A pair carries a *contract*: its assurance class, the direction of its square, its kept observables, its measured cost [4]. Contracts compose along a route by the componentwise meet and are ordered by strength; the meet is what makes the following duality compose.

Definition 2.3 (Required and achieved contracts; the tiers). An implemented pair carries an *achieved* contract, verified from above: evidence establishes *at least this*. A *frontier pair* carries a *required* contract, demanded from below: the join, over the open questions that cite it, of the observables they need kept, the direction and assurance they need, with cost kept as the histogram of their budgets. It also carries its *design evidence* — everything the books know about how to get there (Section 6) — and, where the demanded target is a reasoning language that does not exist, a *hypothetical* endpoint: a language sketch, a typed hole in the graph. The registry thus holds one lifecycle in three tiers:

$$\text{frontier} \longrightarrow \text{registered} \longrightarrow \text{partial} \longrightarrow \text{built},$$

where *frontier* means design unknown, *registered* means a human admitted a design (the brief’s “intended translator” is filled in), and *partial/built* mean an achieved, measured contract. Registering a frontier pair *means* exhibiting a design whose achievable contract dominates the required one; promotion only ever advances, and the evidence payload travels (Theorem 5.7).

Definition 2.4 (Conditional routes). A route may mix implemented and frontier hops. Its *conditional contract* is the meet over all hops — achieved contracts for implemented hops, required contracts for frontier hops — and it is read as: *if* each frontier hop is discharged by a pair achieving its requirement, this route answers q with at least this contract.

Conditional routes are never executable; they are the plan-shaped reading of the map, and Theorem 5.7 is what makes them sound.

Definition 2.5 (Saturation). B is *saturated at G* when the pair-shaped demand its open questions derive contains no *registered-able* candidate over the known set — no tier-2 design over the registered languages, solvers, and anchors would add a feasible way, a cheaper way, or a more trusted way — i.e., every question in B is in one of two terminal states: *solved, all ways*, its answer carrying the census of every feasible route with measured cost profile, assurance class, and corroboration; or *open, on the books*, its frontier pairs (if any are honest to name) all lying outside the known set, partitioned by obstacle. Saturation is an emptiness check on a derived set — mechanically detectable, no judgment call — and it is *reopened only by good news*: a new solver, anchor, or logic re-enters the known set exactly where the terminal board says it plugs in.

The map. The deliverable of saturating B is its *map*: the answered fraction of B per loop iteration (monotone, by Theorem 5.2); cost per answer across iterations; per solved question, the way-census; and the *terminal board* — the surviving frontier pairs with their obstacle partition, required contracts, design evidence, and distinct-question counts. The board is the domain’s future-work section, machine-generated; the map is drawn in the same currency the instrument is built from.

3 The instrument, as means

A trustworthy frontier map needs an instrument with specific properties, and each is supplied by one feature of hurdy-gurdy: a map its untrusted explorer cannot falsify needs determinism and the commuting square; “all ways” needs routes, branches, and fidelity accounting; the local gradient needs the five-obstacle diagnosis; “everything unsolvable, saved” needs the books. This section states each feature in one paragraph, as the requirement it serves.

4 The loop

The explorer runs two loops with different payoffs: a capability loop triggered by unanswerable questions (the diagnosis names the missing item) and a trust loop triggered by answers weaker than the asker’s floor (a second, independently derived route manufactures fidelity). Unmet demand is recorded, aggregated per generation target, and promoted to a registered design only by a human act — or by a human-written, revocable mandate whose mechanical instantiation is delegated. Registered refinements of over-approximating pairs make the cost axis a CEGAR loop whose refinements are reusable instruments rather than solver-internal state.

5 Why this explores the frontier

An explorer of a frontier must satisfy six properties or it is not an explorer: it must draw a map its untrusted explorer cannot falsify (F1), never lose ground (F2), know at every open point what would extend the map there (F3), eventually reach every point inside the true frontier (F4), know when a survey is finished (F5), and work in any territory (F6). This section states each as a theorem over the model of Section 2, with its proof burden placed honestly: mechanized, reduced to a named assumption, or empirical. The model is small on purpose — a registry is a set of items, a candidate is a list of items, and answerability is the filtration of Definition 2.1; its two monotonicities (stages shrink along conditions, grow along registries) carry F2–F5.

Theorem 5.1 (F1, unfalsifiable map). *Whatever the generator of pairs does — blind or adversarial — the map’s labels are truthful: (i) every existential answer is true at the source, assuming only adequacy of the source interpreter; (ii) every universal answer carries exactly the assurance class, direction, and trusted base computed by the componentwise meet of its route’s contracts — labels never overstate, because meets never exceed their arguments.*

This is the instrument’s own theory, consumed as an interface: (i) is witness replay (existential_self_certifying, axiom-free), (ii) is the contract algebra (weakest_link_universal, Contract.comp_glb, lax_universal_transfer) [4]. The residue is stated, not hidden: a false universal in the corner where no branch corroborates and no anchor covers is not excluded by theorem — it is bounded empirically (measured escape rates, negative controls) and shrunk by the trust loop, and the map records per answer which corner it sits in.

Theorem 5.2 (F2, monotone exploration). *If $G \subseteq G'$ then every question answerable at G is answerable at G' (Frontier.answerable_mono, axiom-free); and every standing verdict stands, by the pair-level ratchet (ratchet_preserves_faithful).*

Theorem 5.3 (F3, complete local gradient). *For every question unanswerable at G : a first failing condition exists (diagnosis_total), is unique (diagnosis_unique), never regresses under growth (diagnosis_progress), and strictly advances under any adequate extension — one after which some admitted candidate survives one condition further (diagnosis_strict_progress).*

Totality is where the model earns the word *complete*: the five conditions are exhaustive *by construction*, because answerability is defined as their conjunction (Definition 2.1), so the demand taxonomy is a theorem input, not a design choice. What the platform’s diagnosis names — that the returned generation target, if admitted, discharges the failing condition — is the specification of why_not, met by construction in the implementation; it enters the theorems only as the adequacy hypothesis. The one classical step (diagnosis_total) is itself

informative: an unanswerable question does not name its obstacle constructively — the platform computes it because its five conditions are decidable.

Theorem 5.4 (F4, the chain lemma). *Along any growing chain of registries in which every diagnosis is met by an adequate extension, N steps answer the question (adequate_chain_answerable): the diagnosis index is bounded by N and strictly increases, so it runs out of room.*

Assumption 1 (Fair enumeration). Every demanded target is eventually attempted: the books aggregate demand, and a human — or a scoped mandate — eventually acts on standing demand.

Assumption 2 (Gate liveness). A sound design with adequate evidence is eventually admitted: builders eventually produce, and the gate does not reject forever, a pair discharging a named requirement that some sound pair can discharge.

Theorem 5.5 (F4, relative completeness). *Under Assumptions 1 and 2, every question answerable at some finite extension of G over the candidate universe is eventually answered by the loop.*

The mathematical content is Theorem 5.4; the assumptions supply its chain, and they are the theorem’s honest trusted base — both are what the automation pipeline exists to provide, and both are measured (queue throughput, gate escape rates) rather than provable. The framing is deliberately that of relative completeness [2]: complete relative to the candidate universe and the domain’s supply, not an oracle claim — undecidability, the adequacy floor of the source semantics, and the finite anchor supply stand at every registry size.

Theorem 5.6 (F5, saturation is a decidable, reachable fix-point). *For a pinned finite benchmark and a finite known set, the saturation predicate of Definition 2.5 is computable from the books — an emptiness check on the derived tier-2 set — and the loop restricted to in-set targets reaches it in finitely many iterations: target signatures are finitely many, an admitted signature cannot recur (dedup plus the ratchet), and each iteration retires at least one.*

The finite-pool argument is mechanized (saturation_terminates): admitted signatures never recur and form a duplicate-free subset of the pool, so admissions run out of room. The decidability half is the implementation’s contract: saturation is an exit code, not a judgment.

Theorem 5.7 (Currency: the status ratchet; conditional plans). *(i) Along any lifecycle a pair’s tier only advances and its evidence payload only accumulates (lifecycle_ratchet, axiom-free): a built pair still carries the demand that created it. (ii) A mixed route’s conditional contract (Definition 2.4) is a lower bound on its realized contract once every frontier hop is discharged by a pair whose achieved contract dominates its*

requirement (conditional_plan_sound, via monotonicity of the meet): *conditional plans never overpromise*.

Theorem 5.8 (F6, domain-genericity). *F1–F5 are stated and proved over the domain signature alone — abstract languages, items, conditions, contracts — so they transfer to any domain by instantiation. What does not transfer for free is the supply side, and Section 6 states it as the explicit obligation: the theorems are conditional on the kit, and the books measure the condition.*

Necessity: the ablations. Each load-bearing feature of the instrument is needed for a specific theorem, and dropping it breaks that theorem — which is the precise sense in which the current platform is the means to this paper’s end. Without *determinism*, a square check cannot distinguish defect from coin flip and F1(i)’s replay proves nothing. Without the *ratchet*, F2 fails and the curve can regress; a map that can silently shrink is not a map. Without *typed aborts* and the fixed diagnosis order, the filtration of Definition 2.1 has no well-defined first failure and F3 degenerates to blind enumeration. Without the *existential/universal asymmetry* — replay for witnesses, graded meets for universals — an untrusted generator forges map entries and F1 falls adversarially. Without *pinned ingestion*, “the benchmark” is not a fixed set and F5’s fixpoint is not defined. And without the *human valve* (or its mandate generalization, which preserves it), Assumption 1’s enumeration has no scope and the books no meaning: demand-driven growth presupposes someone accountable for what counts as demand.

The trusted base, collected. F1(i) assumes source-interpreter adequacy; F1(ii)’s corner is empirical; F4 assumes Assumptions 1 and 2; F5 assumes the pin and the finiteness of the known set; F6 defers the supply side to the kit. Nothing else is assumed, and each assumption is either measured by the platform’s own books or discharged by construction in the implementation — the reader can locate every claim’s burden, which is the property the whole architecture is built to have.

6 The domain kit, and the far side as a design oracle

Pointing the explorer at a domain costs four things: a reasoning hub with a mechanized decision procedure and declared budgets; deterministic shared interpreters for each language touched; at least one external semantic anchor, if trust beyond per-run checking is wanted; and a pinned benchmark. Given the kit, the theorems of Section 5 apply verbatim — and the terminal board the loop leaves behind is not a failure list but a design oracle: each obstacle class carries an extraction operator turning its open questions’ evidence into the specification of the missing instrument, exported as frontier pairs — self-contained challenge bundles for the domain’s solver community.

7 Related work

[1–5]

8 Limitations and conclusion

Four walls stand at every registry size — undecidability, the adequacy floor of the source semantics, worst-case decision cost, and the finite supply of independent anchors — and the theorems of Section 5 are drawn up to them, not past them. Within the walls, saturation makes “we don’t know how to decide this” a typed, clustered, cost-profiled specification; the maps compound, because pairs are shared.

Acknowledgments

This work was co-funded by the Czech Science Foundation under Grant No. 23-07580X and the European Union under the project Robotics and Advanced Industrial Production (reg. no. CZ.02.01.01/00/22_008/0004590).

References

- [1] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. Counterexample-Guided Abstraction Refinement. In *CAV (LNCS, Vol. 1855)*. 154–169.
- [2] Stephen A. Cook. 1978. Soundness and Completeness of an Axiom System for Program Verification. *SIAM J. Comput.* 7, 1 (1978), 70–90.
- [3] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *POPL*. 238–252.
- [4] Christoph Kirsch. 2026. Untrusted Authors, Trusted Answers: A Calculus of Fidelity-Graded Translations. arXiv preprint, v3. Code, evidence, and the Lean mechanization: <https://github.com/cksystems/hurdy-gurdy>.
- [5] Robert P. Kurshan. 1994. *Computer-Aided Verification of Coordinating Processes: The Automata-Theoretic Approach*. Princeton University Press.